DEPARTMENT OF SMART COMPUTING AND CYBER RESILIENCE

FACULTY OF ENGINEERING AND TECHNOLOGY

FINAL PROJECT FOR CSC1024\PRG3024\FEL1184 PROGRAMMING PRINCIPLES

ACADEMIC SESSION: May 2026 DISTRIBUTION: WEEK 06

SUBMISSION DEADLINE: **2nd August 2026 (Sunday), before 11:59 PM**.

Student Names with IDs:

1. Muhammad Abbas **(26052019)** *(Group Leader)*
2. Eeh Cheng Zher **(25023896)**
3. Gam Yao Teng **(25037318)**
4. Chin Keen Yen **(25027962)**

INSTRUCTIONS TO CANDIDATES

- This programming project will contribute 50% to your final grade.
- This programming project is a final examination.
- This is a group project with a maximum of 5 members.
- The presentation will be carried out in Week 13 during lab slots (Please email your lab instructor to book your presentation slot)
- The group leader will submit the following materials via elearn:
  - ✓ Report in .pdf
  - ✓ Contribution log signed by each member of the group
  - ✓ Program source file in either .py or .ipynb, including .txt files
  - ✓ Link to the recorded presentation. Make sure the accessibility is set to “Everyone”.
  - ✓ Presentation deck in .pdf
- Please ensure that for both report and presentation deck submitted, each student name and ID are included. Also, do note that marks for the presentation may differ from each member based on their understanding and contribution to the project.

IMPORTANT

The University requires students to adhere to submission deadlines for any form of assessment. Penalties are applied in relation to unauthorized late submission of work.

- Project submitted after the deadline but within 1 week will be accepted for a maximum mark of 40%.
- Work handed in following the extension of 1 week after the original deadline will be regarded as a non-submission and marked ZERO.

Lecturer's Remark (Use additional sheet if required)

We, the students as presented above, have received the programming project and read the comments.

Abbas 24/7 Keen 24/7 Yao Teng, 24/7 Cheng Zher 24/7

(Student signatures and Date)

Academic Honesty Acknowledgement

We, the students as presented above, verify that this paper contains entirely our own work. We have not consulted with any outside person or materials other than what was specified (such as an interviewee) in the assignment or the syllabus requirements. Further, we have not copied or inadvertently copied ideas, sentences, or paragraphs from another student. We understand the penalties (as outlined in the student handbook) for any form of copying or collaboration on any assignment.

Abbas 24/7 Keen 24/7 Yao Teng, 24/7 Cheng Zher 24/7

(Student signatures and Date)

Table of Contents

Section 1. (Main Menu)

Introduction

The backbone of the entire program, the main menu is a function and a while True loop which contains the starting menu of choices: Add **New Post**, **Update Post Status**, **Record Engagement**, **Display Content Calendar**, **Generate Performance Report**, **Export Report**, and **Exit**. The whole program, along with the main menu, utilises the 'def' feature of python. Using functions, large sections of code can be called with a single input; as opposed to using a lengthy if/elif statement which would complicate the entire program. Since the main menu function is at the top of the entire program, all other functions must first go through it. Each option from the main menu additionally has its own 'sub-functions', match cases were used to determine what 'sub-function' to run based on the user's input. Match cases were used in place of if/elif statements here for better code readability.

Challenges and Solutions

When we began the project, functions had not been taught in class yet. Furthermore, the upcoming lecture slides were also not uploaded so there was no possibility to refer directly to our lecturer's slides. As a result, we had to utilise the W3schools website to study the usage of functions along with hands-on learning examples.

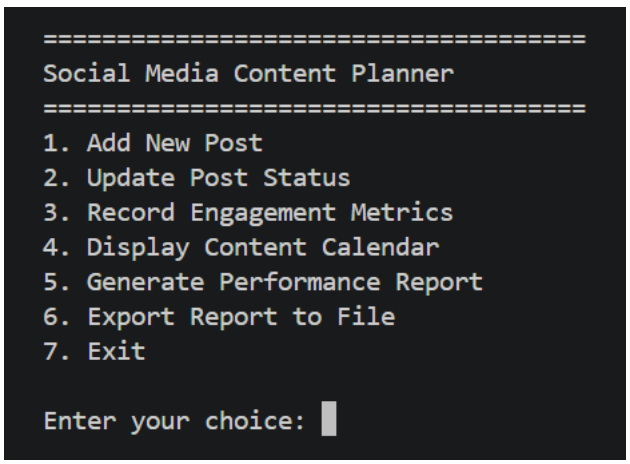
Strengths and Limitations

Since the entire program, including the main menu, utilises functions instead of if/elif statements it allows it to remain separate and organised throughout. Furthermore, using match cases inside the functions keeps it simple when adding new options/features to each function. Finally, the main menu is the function which all other functions are called through, ensuring that the program flow is easily legible. Despite that, the program uses hardcoded number (1-7) as the way for users to pick the options/features that they want to use, and adding newer options would cause them to shift accordingly. In example, in our current program '7' is the number for Exit, however, if a new option were to be added then it would subsequently become '8'. The program additionally does not confirm with the user before it terminates.

Future Suggestions

A confirmation input (y/n) should be added before allowing the user to completely exit the program. An additional option in the main menu could be added which explains how each option works, allowing new users to grasp the features easier. Could add additional feedback for when user inputs invalid data type (e.g. "Please enter a number" or "Entered number is not within the range").

Command Line Output



Section 2. (Adding New Posts)

Introduction

Allows users to add/record a new post for their social media platform. It contains the functions of `new_post_menu()` and `draft_new_post()`; `new_post_menu()` is the submenu which displays the options: **Draft New Post**, **Go To Main Menu**, or **Exit**. `draft_new_post()` is the function which accepts the user's input to create a new post entry. Both **"`file_check("posts.txt")`"** and **"`file_check("platforms.txt")`"** will be called to check whether both files exist. If they do not exist, the function will automatically create them while also populating `platforms.txt` with valid platforms. The user enters the correct match case to add a new post through the main menu, a sub-menu is then created to allow creation of a new post which is stored with 5 columns in a comma separated single line in the `posts.txt` file following this format: **post_id**, **platform**, **caption**, **schedule_date**, **status**. The user is prompted to create a new Post ID which is first verified in `posts.txt` to ensure that no duplicates exist. User is then asked to input valid choice of platform from the available list in `platforms.txt`. Caption and post schedule date are then accepted, completing the drafting of a new post. Finally, the post is appended into the `posts.txt` file.

Challenges and Solutions

Post IDs are saved as plain text, possibly leading to overlaps in IDs unless they are check prior. As such, the `draft_new_post()` function first checks whether the entered post ID already exists within the `posts.txt` file before continuing. If it already exists then it will return back to the main menu loop directly. Besides that, platforms were added into a separate text file to store all of the available or supported platforms as opposed to being coded straight into the program. If the platforms were hardcoded in the program, the entire source code would need to be updated consistently.

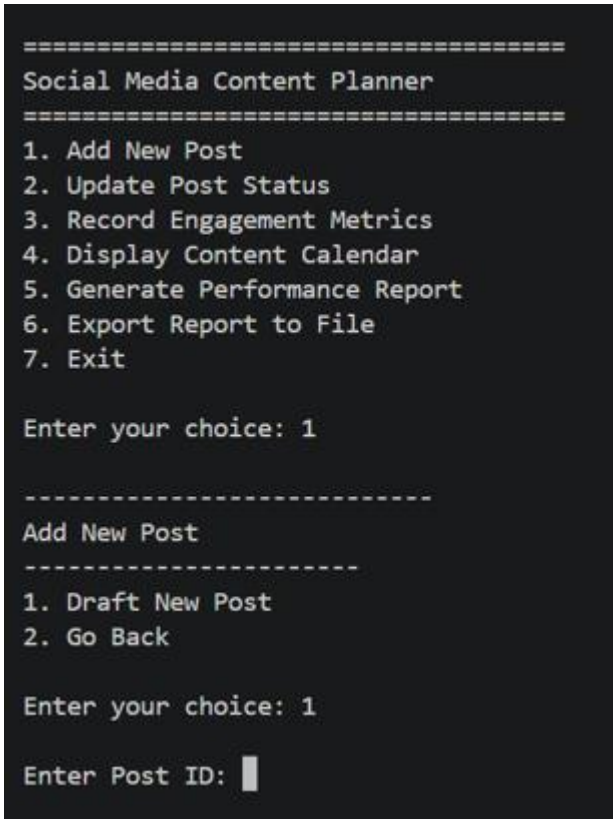
Strengths and Limitations

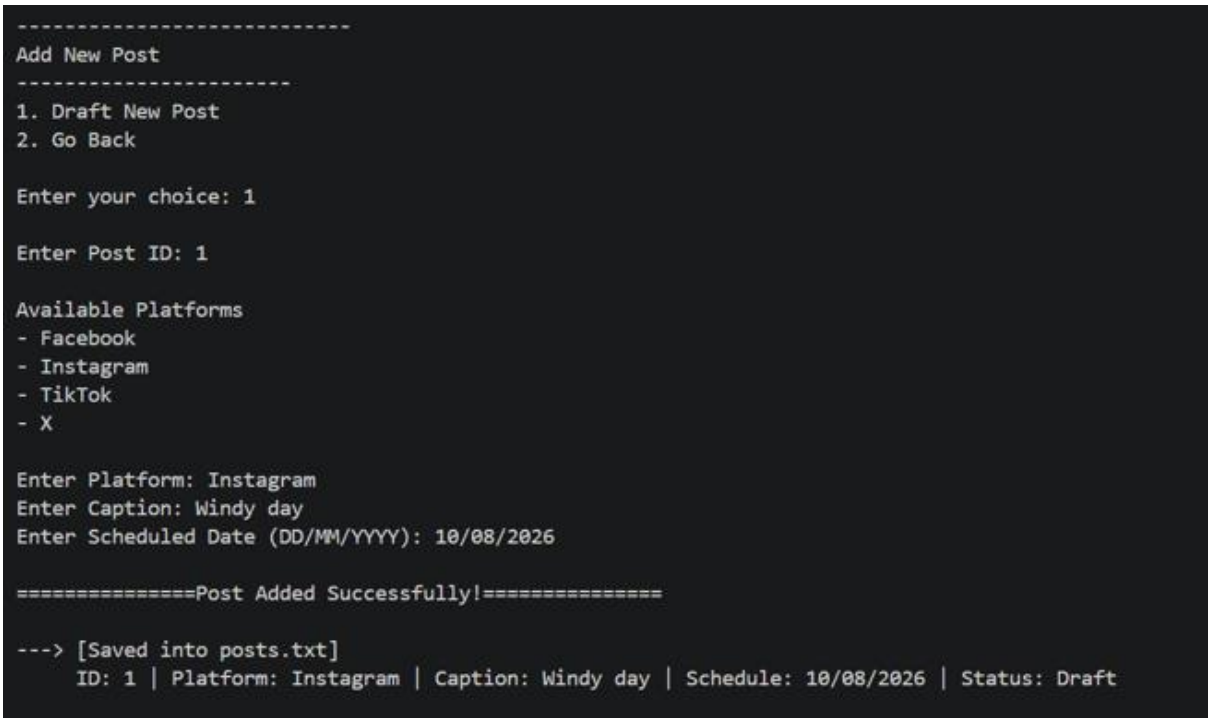
The `draft_new_post()` function prevents duplicates of post IDs immediately after being called. New platforms that are supported can easily be added by updating the `platforms.txt` file alone, no need to alter the current code. However, it does not have a preview or confirmation of the draft post which displays the user's input summary.

Future Suggestions

Add a confirmation of the newly drafted post which includes a brief summary display of the related details. The date of the newly drafted post should be checked that it is not in the past. Platform input could be done with numbered options like other menus instead of typing out the whole platform name.

Command Line Output


Social Media Content Planner
1. Add New Post
2. Update Post Status
3. Record Engagement Metrics
4. Display Content Calendar
5. Generate Performance Report
6. Export Report to File
7. Exit
Enter your choice: 1
Add New Post
1. Draft New Post
2. Go Back
Enter your choice: 1
Enter Post ID:


Add New Post
1. Draft New Post
2. Go Back
Enter your choice: 1
Enter Post ID: 1
Available Platforms
- Facebook
- Instagram
- TikTok
- X
Enter Platform: Instagram
Enter Caption: Windy day
Enter Scheduled Date (DD/MM/YYYY): 10/08/2026
==========Post Added Successfully!==========
---> [Saved into posts.txt]
ID: 1 | Platform: Instagram | Caption: Windy day | Schedule: 10/08/2026 | Status: Draft

Section 3. (Updating Post Status)

Introduction

Existing posts can be updated and pushed to a different status with this function. It contains the `post_status_menu()` and `update_post_status()` functions. `post_status_menu()` creates the sub-menu to display the options for the user: "**Update Status**", "**Go To Main Menu**", and "**Exit**"."**Go To Main Menu**" returns to the main menu loop and prints the list of options again, while "**Exit**" ends the program completely. If "**Update Status**" is selected, then `update_post_status()` function is called, and the user is asked to enter the Post ID to be updated. If Post ID is not able to be found then "No posts to be updated, create a post and try again." is printed, returning to `post_status_menu()` and the file does not get modified.

If the post is already marked as "Posted" in status, then no changes can be made to it. Changes can only be made if the post is either in "Draft" or "Scheduled" status. Unknown statuses are caught with an else statement, any other status than "Draft", "Scheduled", or "Posted" will print "Post Status Unknown... Unable to update.". Posts that are in "Draft" status can be changed to "Scheduled" or directly into "Posted", while "Scheduled" posts can be progressed to "Posted" with a (y/n) prompt but may not be converted back to "Draft". The entire `posts.txt` is then rewritten, with the selected Post ID lines being replaced to update with the new post status. Unselected Post IDs will remain unchanged as they are stored in a separate list to be rewritten back into the `posts.txt` afterwards.

Challenges and Solutions

Initially attempted to create the `update_post_status()` function by starting with opening the `posts.txt` file in write mode. This caused the entire file to be deleted before the specific post ID could be found since write mode rebuilds the text file from scratch. Instead, the prior version of `posts.txt` was read into memory with `file.readlines()` and created a new list called `new_lines[]` to store the newly updated version of the posts list which will get stored into the "posts.txt" file. The only line that will change is the line with the specific post ID that the user entered to update post status for, while the other lines remain unchanged. For proper updates to be made to the post status, the current status of each post must be checked and maintained properly to ensure the flow follows as intended. After reading `posts.txt` into memory, it ensures that each line in the file is split with commas and stripped of excess whitespace to section it out into: **Post ID, Platform, Caption, Scheduled Date, Current Status**. Given that the post ID the user entered exists, the program will check the status of the post and updates it accurately into the `data[]` list at index 4. All updates to the post status relies on `data[4]` as it stores each post's current status.

Strengths and Limitations

The entire "posts.txt" file is read before any attempts to write are made. This prevent any possibilities for the post list to be lost. All posts begin at the "Draft" status which ensures that the post status update flow is always adhered to accurately. Apart from that, if there ever exists a post with a status other than the defined "Posted", "Scheduled", and "Draft" then it is caught by an else statement and cancels the update to the post status. Although, the current method of storing posts uses commas to split each field/column. If any of the columns for a post were to have a comma then the program would no longer function as intended because the fields are aligned to different/inaccurate items.

Future Suggestions

If the entered post ID is not found, the code should have it display immediately after checking the post's current status. The current code has the file be checked in its entirety, going through all of the if

and elif statements for post status before returning that the post ID could not be found/matched. Scheduled posts could be allowed to return back to draft since there is a possibly that the user accidentally shifted the post status. Additional confirmations (y/n) could be added for other stages instead of just for Scheduled to Posted.

Command Line Output (Continuation on pg. 9)

```
===============Update Post Status===============

Enter the Post ID to update: 1

Post ID: 1
Platform: Instagram
Caption: Windy day
Scheduled Date: 10/08/2026
Current Status: Scheduled

Update status to Posted? (y/n): y
Status has been updated to Posted.
```

```
===============Update Post Status===============

Enter the Post ID to update: 1

Post ID: 1
Platform: Instagram
Caption: Windy day
Scheduled Date: 10/08/2026
Current Status: Posted

This post has already been posted, no updates can be made.
```

```
====================================
Social Media Content Planner
====================================
1. Add New Post
2. Update Post Status
3. Record Engagement Metrics
4. Display Content Calendar
5. Generate Performance Report
6. Export Report to File
7. Exit

Enter your choice: 2

---------------------------
Update Post Status
----------------------
1. Update Status
2. Go back

Enter your choice: 1

==============Update Post Status==============

Enter the Post ID to update:
```

```
==============Update Post Status==============

Enter the Post ID to update: 1

Post ID: 1
Platform: Instagram
Caption: Windy day
Scheduled Date: 10/08/2026
Current Status: Draft

1. Update to Scheduled
2. Update to Posted
3. Cancel
Enter option (1-3)
1
Post Status has been updated to Scheduled.

---------------------------
Update Post Status
----------------------
1. Update Status
2. Go back

Enter your choice:
```

Section 4. (Recording Engagement)

Introduction

This function allows for engagement metrics to be logged into `engagement.txt`. This function utilises `engagement_entry()` as the main function, with the sub functions: `posted_check()`, `posts_data_extraction()`, `id_verification()`, and `validate_positive_integer()`.

After the main function is called, `"file_check("engagement.txt")"` will first check if `engagement.txt` file exists with a `try-except` block. If it does not already exist then it will automatically be created for the user. Next, the main function calls `posted_check()` which will read through `posts.txt` and return all of the lines in the form of a list which only includes posts that are in the "Posted" status; the "Posted" status itself will be omitted. Next, the entire list of valid posts will be printed out. The main function then asks the user to input the post ID which they want to record. The entered post ID will get verified by `id_verification()` which checks the existing `engagement.txt` file for any duplicates of the post ID. Given that there is a duplicate, the user will need to enter a different post ID until there is no duplicates.

Once verified, `posts_data_extraction()` will read through the list from `posted_check()` to return the data related to the entered post ID. A sentinel loop is used to allow the user to exit the function by entering the letter 'Q'. The user will then be asked to input the number of views, likes, comments, and shares for the selected post by `engagement_entry()`. The input gets directed into `validate_positive_integer()` within the `validation.py` file. This function ensures that the input is a either a positive integer or zero. The data from `posts_data_extraction()` and the engagement values from the user will be appended into the `engagement.txt` file, separated with commas for each field. The main function will print a success message and displays all of the recorded data before returning to the main menu.

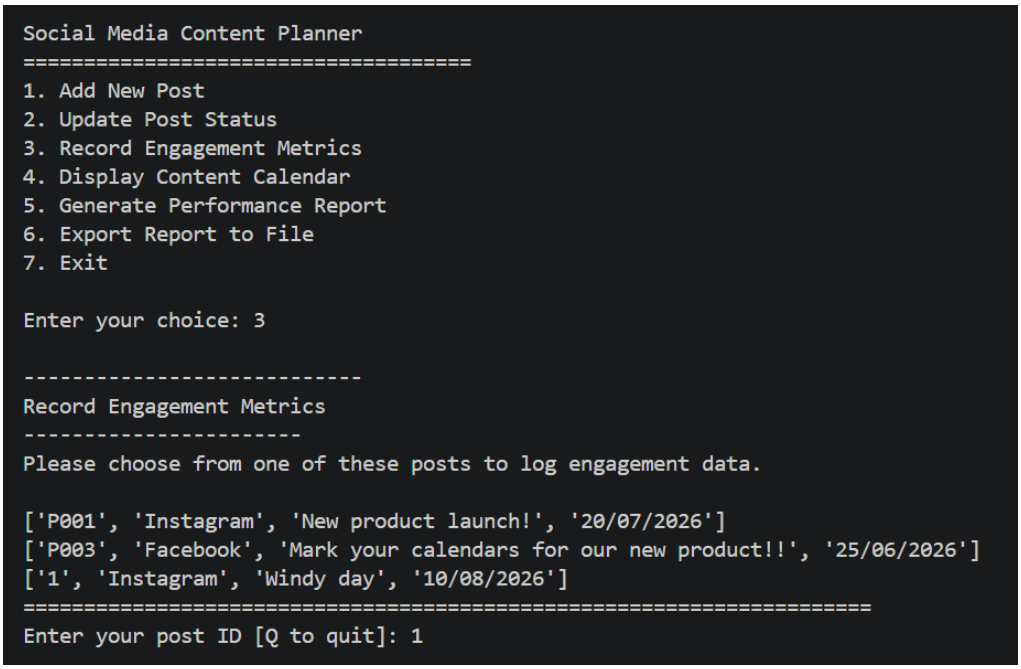
Challenges and Solutions

The user may not have the `engagement.txt` file before using the program which would lead to the program crashing or ending prematurely. To combat this, a try except block was used. The `engagement.txt` file will first be read, if it does not currently exist (`FileNotFoundError`) then the file will automatically be created. Apart from that, users may enter incorrect data types (especially for int) which causes the program to either crash or save the wrong data type. A function was created for easier access during conversion of the user input into an integer.

The extraction of the correct data column of post status from `posts.txt` was an issue when attempting to check whether a post ID is currently in the "Posted" status. A for loop was utilised to check through each line in `posts.txt` to identify `data[4]` which is the current post status, returning the data as required. This post status check allowed for less memory usage, preventing the need to recheck the whole file again since the status of the entered post ID will be compared against the status check and returned accordingly.

Finally, if the `engagement.txt` file were to be empty, the variables used in the next lines will not be assigned, causing the program to crash. The `posts_data_extraction()` function returns and displays an error message instead of the program crashing when the user input does not match any of the post IDs.

Strengths and Limitations



The `engagement.txt` file is automatically checked and created for the user if it does not already exist. Platforms are automatically identified and filled in with just the post ID and the user's input data type is automatically verified to prevent the program from crashing unexpectedly. The list of all valid posts and their respective/relevant details will be printed when attempting to record the engagement matrix. Post engagement matrix that were logged prior are not able to be updated and it does not allow the user to exit once they have entered a valid post ID.

Future Suggestions

The current program can only enter new entries and cannot update previous entries. This is due to the program checking whether an entry of said post ID is already inside the file. In the future, adding an option to update engagement metrics is much needed. While the program allows the user to abort the process, it does not allow the user to abort after entering a valid post ID. This prevents the user from correcting their mistakes (entering wrong value) without manually terminating the program. In the future, a sentinel loop should be included during engagement metrics entry that allows the user to abort whenever they want.

Command Line Output

```
---------------------------
Record Engagement Metrics
----------------------
Please choose from one of these posts to log engagement data.

['P001', 'Instagram', 'New product launch!', '20/07/2026']
['P003', 'Facebook', 'Mark your calendars for our new product!!', '25/06/2026']
['1', 'Instagram', 'Windy day', '10/08/2026']
=====================================================================
Enter your post ID [Q to quit]: 1

============================================================
This ID already has a record, please enter another post ID.
Enter your post ID [Q to quit]: P001

You are currently adding entry for P001.
Enter the number of views: 1234
Enter the number of likes: 123
Enter the number of comments: 23
Enter the number of shares: 12

=======================================
Engagement data successfully logged.

PostID, Platform, Views, Likes, Comments, Shares
P001, Instagram, 1234, 123, 23, 12
```

Section 5. (Display Content Calendar)

Introduction

The content calendar of the planner can be accessed through the Main Menu by selecting option 4. Once selected, it uses the `notification()` function to display "Content Calendar". It then displays a table containing five columns: Post ID, Platform, Caption, Date (scheduled or posted), and Status (Draft, Scheduled, or Posted). The function reads all saved posts from the posts.txt file and formats them into a clear table, making it easier for users to view and manage all of their content in one place.

Challenges and Solutions

During the group project, we initially overlooked the requirement for the content calendar to display posts in order of their scheduled date, and this was only discovered when reviewing the completed program. Since the posts were originally displayed in the order they were created, we needed to modify the function so that it sorted the data before displaying it. To solve this, the post information was first read from the file, organised based on the scheduled date, and then displayed in the correct order. This ensured that users could easily view upcoming posts in chronological order and that the program met the project requirements.

Strengths and Limitations

One of the strengths of the `display_content_calendar()` function is its robustness and clear readability. It uses `try-except` blocks to safely handle situations where the `posts.txt` file cannot be found, preventing the program from crashing unexpectedly. If the file is missing or empty, the function informs the user instead of attempting to process invalid data. Another strength is the way the output is formatted into a structured table with aligned columns, making the content calendar much easier to read. Long captions are also shortened with trailing fullstops (eg. Long name....) to prevent text from overflowing into other columns and affecting the table layout.

But it still assumes that every line in the `posts.txt` file is correctly formatted with the required fields. If the file becomes corrupted or contains missing data, the function may not display the information correctly. It also uses commas to separate the stored data, meaning captions containing commas could be split incorrectly. In addition, the function only displays the stored posts and does not allow users to search, filter, or sort the content using different options.

Future Suggestions

The content calendar could be improved by allowing users to filter posts based on their platform or status, making it easier to find specific content. A search feature could also be added to quickly locate posts by their caption or Post ID. Another improvement would be to automatically validate the data stored in the `posts.txt` file before displaying it, reducing the chance of errors if the file is modified or corrupted. Finally, the content calendar could support additional sorting options, such as sorting by platform or post status, to provide users with greater flexibility when managing their scheduled content.

Command Line Interface (pg. 14)

```
6. Export Report to File
7. Exit

Enter your choice: 4

==============Content Calendar==============

Post ID   Platform      Caption                 Date         Status
--------------------------------------------------------------------------
38        TikTok        New product launch!     20/12/2026   Posted
78        Facebook      Company update          20/12/2026   Scheduled
21        Instagram     Customer testimonial    10/12/2026   Posted
100       Facebook      New product launch!     26/11/2026   Draft
68        Instagram     Customer testimonial    25/11/2026   Posted
87        Facebook      Company update          24/11/2026   Posted
34        X             Flash sale              23/11/2026   Scheduled
36        Instagram     Giveaway alert          19/11/2026   Posted
51        Facebook      Behind the scenes       17/11/2026   Draft
94        Instagram     Giveaway alert          11/11/2026   Posted
17        X             Company update          08/11/2026   Draft
55        Facebook      Holiday special         28/10/2026   Posted
26        X             Flash sale              26/10/2026   Draft
63        Facebook      Giveaway alert          25/10/2026   Draft
31        Facebook      Industry news           13/10/2026   Posted
37        Instagram     Flash sale              11/10/2026   Draft
3         Instagram     New product launch!     05/10/2026   Draft
50        TikTok        New product launch!     04/10/2026   Posted
13        Facebook      Flash sale              28/09/2026   Draft
82        TikTok        Giveaway alert          24/09/2026   Posted
29        X             Behind the scenes       21/09/2026   Draft
53        TikTok        Did you know?           20/09/2026   Posted
64        X             Weekly recap            16/09/2026   Scheduled
93        X             Industry news           14/09/2026   Scheduled
15        X             Industry news           13/09/2026   Draft
35        X             Community spotlight     13/09/2026   Posted
```

Section 6. (Generate Performance Report)

Introduction

The Performance Report can be accessed from the Main Menu and provides users with a summary of their social media performance. After selecting the report option, users are presented with a submenu where they can choose to view the total number of posts, the best performing post, the highest interacted platform, or print all reports together. The report uses data stored in the `engagement.txt` file to calculate and display performance statistics in a clear and organised format.

Challenges and Solutions

One challenge during development was creating a fair way to compare the performance of different posts. Since views, likes, comments, and shares have different levels of importance, simply adding the values together would not produce meaningful results. To solve this, a weighted scoring formula was made, giving each engagement metric a different value when calculating the overall performance score. Another challenge was handling situations where the engagement data had not yet been created. This was resolved by using `try-except` blocks and checking for empty data, allowing the program to display a helpful message instead of terminating unexpectedly.

Strengths and Limitations

A strong point is the use of a weighted performance score, which considers multiple engagement metrics instead of relying on only one, giving a more balanced evaluation of each post. The report also supports ties when identifying the highest interacted platform, allowing multiple platforms to be displayed if they achieve the same score.

However, the report also has some limitations. The calculations depend entirely on the accuracy of the data stored in the `engagement.txt` file, so incorrect or incomplete data will produce inaccurate results. The weighting values used in the performance score are fixed and may not accurately reflect every user's priorities or marketing goals. In addition, the report only provides a summary of the data and does not include graphs or visualisations that could make performance trends easier to understand.

Future Suggestions

The Performance Report could be improved by allowing users to customise the weighting used for each engagement metric based on their own objectives. Visual charts, such as bar or pie charts, could also be added to help users better understand their performance across different platforms. Finally, additional report options, such as filtering by date range or comparing platform performance over time, would provide users with more detailed insights into their content performance.

Command Line Interface (pg. 16)

```
---------------------------
Performance Report
----------------------

1. Total Posts
2. Best Performing Post
3. Highest Interacted Platform
4. Print All
5. Return

Enter your choice: 1


Total Posts From All Platforms
==============================
Facebook -- 11
Instagram -- 9
TikTok -- 13
X -- 3
Total Posted -- 45
==============================
```

```
Enter your choice: 2


Best Performing Post
==============================
Post ID -- 36
No. Views -- 12566
No. Likes -- 1604
No. Comments -- 300
No. Shares -- 59
Performance Score -- 55.49
==============================
```

```
Enter your choice: 3


Highest Interacted Platform
==============================
Platform -- TikTok
Total Views -- 76689
Total Likes -- 10775
Total Comments -- 1368
Total Shares -- 930
Interaction Score -- 106.50
==============================
```

```
5. Return

Enter your choice: 4


Total Posts From All Platforms
==============================
Facebook -- 11
Instagram -- 9
TikTok -- 13
X -- 3
Total Posted -- 45
==============================


Best Performing Post
==============================
Post ID -- 36
No. Views -- 12566
No. Likes -- 1604
No. Comments -- 300
No. Shares -- 59
Performance Score -- 55.49
==============================



Highest Interacted Platform
==============================
Platform -- TikTok
Total Views -- 76689
Total Likes -- 10775
Total Comments -- 1368
Total Shares -- 930
Interaction Score -- 106.50
==============================



---------------------------
Performance Report
----------------------
```

Section 7. (Export Report)

Introduction

The Export Report feature makes it possible for users to save the generated Performance Report as a text file. Before exporting, the function checks whether the required data is available. Users are then prompted to enter a file name, after which the report is generated and saved as a `.txt` file in the program directory. This gives users to keep a copy of the report for future reference or sharing.

Challenges and Solutions

One challenge in developing this feature was ensuring that users cannot export the report if there is no engagement data. Without validation, the program would either generate an incomplete report or produce an error. To solve this, we made it so that the function first checks whether the `engagement.txt` file exists before continuing. Another challenge was handling invalid user input when entering a file name. This was resolved by repeatedly prompting the user until a valid file name was entered or the operation was cancelled by entering **Q**.

Strengths and Limitations

One strength of the `export_report()` function is its user-friendly input validation. It prevents users from entering an empty file name and provides the option to cancel the export at any time. The function also uses exception handling to detect whether the required engagement data exists before attempting to generate the report, preventing the program from crashing unexpectedly. Another advantage is that it combines the three individual performance reports into a single text file, making it far better than storing in 3 separate files.

But there are still some drawbacks. The exported report is only saved as a plain text and may not be suitable for professional presentation. The function also overwrites an existing file if the user enters the same file name, which could result in accidental data loss. In addition, the report is always saved to the program's current directory, meaning users cannot choose to save the file in their desired destination.

Future Suggestions

The export feature could be improved by allowing users to select the folder where the report will be saved. Support for additional file formats, such as PDF or CSV, could also make the report more suitable for sharing and presentation. Finally, the function could also run a validation check on if there already exists a file with the same name and ask the user whether they would like to overwrite it or choose a different file name.

Command Line Interface (pg. 18)

```
5. Generate Performance Report
6. Export Report to File
7. Exit

Enter your choice: 6

---------------------------
Exporting Performance Report..
----------------------

Enter the name of the file (Don't include file extensions) [Q to quit]: perfReport


Performance Report successfully exported as perfReport.txt
================================================================


===================================
```

aMain.py | perfReport.txt | posts.t

perfReport.txt

```
Total Posts From All Platforms
==============================
Facebook -- 11
Instagram -- 9
TikTok -- 13
X -- 3
Total Posted -- 45
==============================

Best Performing Post
==============================
Post ID -- 36
No. Views -- 12566
No. Likes -- 1604
No. Comments -- 300
No. Shares -- 59
Performance Score -- 55.49
==============================


Highest Interacted Platform
==============================
Platform -- TikTok
Total Views -- 76689
Total Likes -- 10775
Total Comments -- 1368
Total Shares -- 930
Interaction Score -- 106.50
==============================

```

CSC1024 Programming Principles

Final Project

Python-Powered Social Media Content Planner

Project Description:

Social media is central to Gen Z life, from personal branding to content creation. This project challenges you to build a Social Media Content Planner, a text-based application that helps content creators plan, track, and analyse their posts across multiple platforms. The system will manage post ideas, schedule content, track engagement metrics, and generate performance summaries, all powered by Python and plain text files.

Project Requirements:

a. Initial Data Preparation: Create the following text files to serve as the system's persistent data storage:

- ✓ posts.txt – Fields: Post ID, platform (e.g. Instagram, TikTok, X), content caption, scheduled date, status (Draft/Scheduled/Posted).
- ✓ platforms.txt – Fields: Platform ID, platform name, follower count.
- ✓ engagement.txt – Fields: Post ID, likes, comments, shares, views.

b. Program Functionality: Implement a user-friendly menu system with the following options (at least):

- ✓ Add a new post idea: Allow the user to draft a new post by entering the Post ID, target platform, content caption, and scheduled date. Status is set to Draft by default.
- ✓ Update post status: Enable the user to change the status of a post from Draft to Scheduled, or from Scheduled to Posted.

✓ Record engagement metrics: Allow the user to log engagement data (likes, comments, shares, views) for a posted piece of content.

✓ Display content calendar: Show all posts sorted by scheduled date, displaying the platform, caption preview, and current status.

✓ Generate performance report: Display a summary including total posts per platform, the best-performing post (highest engagement), and the platform with the most interaction.

✓ Export report to file: Save the performance summary to a new text file (e.g. report.txt) for sharing or record keeping.

✓ Exit: Provide an option to exit the program gracefully.

c. Programming Techniques: Your solution must demonstrate the following programming techniques:

✓ Use Command Line Interfacing.

✓ Use input and display functions for user interaction and data input/output. ✓ Employ file handling for data persistence and retrieval.

✓ Use conditional statements and loops for decision-making and repeated tasks. ✓ Define functions for modularity and reusability.

d. Input and Output Handling Requirements: Your program must handle invalid or unexpected user input without crashing and must produce clear, correct, and meaningful output. Every menu option, data entry point, and displayed result should include appropriate validation, error handling, and formatting. Students are expected to think critically about possible input errors, edge cases, incorrect outputs, and misuse scenarios, and design solutions to prevent failures and ensure smooth program execution.

Marking Rubrics (Total Marks: 100):

Group Evaluation (70%)

Assessment Criterion	Marks
Code Organisation and Readability (5 marks) Proper indentation and consistent coding style **2** Meaningful variable and function names **1** Appropriate use of comments to enhance code understanding **1** Logical and efficient code structure **1**	
Functionality and Correctness (25 marks) Accurate implementation of all main functionalities **5** Correct data validation and logical operations **5** Proper handling of updates and deletions **10** Display of information with accuracy and clarity **5**	
Data Persistence and File Handling (10 marks) Proper creation and utilisation of text files for data storage **5** Accurate reading and writing of data to and from files **5**	

User Interface and User Experience (8 marks)

User-friendly menu system **2** Clear and concise prompts and messages **2** Smooth navigation and flow of the program **4**

Error Handling and Robustness (8 marks)

Appropriate error handling for input validation **4** Graceful handling of edge cases and exceptions **2**

Assessment Criterion Marks

Overall program stability and reliability **2**

Flowchart (4 marks)

Clarity and readability of the flowchart **1** Accurate representation of the program flow **1** Inclusion of all major processes and decision points **1** Proper use of flowchart symbols and conventions **1**

Report (10 marks)

Clarity and organisation of the report **2** Detailed explanation of the system design and architecture **2** Discussion of the implementation challenges and solutions **2** Analysis of the system's strengths and limitations **2** Suggestions for future improvements and enhancements **2**

Individual Evaluation (30%)

Assessment Criterion	Marks
Presentation (10 marks) Clarity and effectiveness of the presentation **2** Demonstration of the system's functionality **3** Explanation of the key features and design decisions **3** Time management and adherence to the allotted presentation duration **2**	
Contribution (20 marks) Code Contribution: Student's actual code ownership, logic accuracy, and integration	**5**
Understanding of Code: Ability to explain and justify written code during viva/reflection	**10**
Team Collaboration: Participation, communication, and contribution logs **4** Professionalism & Responsibility: Commitment, documentation effort, and ethics **1**	